

Pricing American Put Options with Least-Squares Monte Carlo: A Study of Longstaff–Schwartz and Tsitsiklis–Van Roy

Ethan, Ranit, Josh

2026-04-07

Table of contents

1	Introduction	1
2	Problem setup: American option pricing as an optimal stopping problem	2
3	Risk-neutral model and benchmark pricing framework	3
4	Longstaff–Schwartz (2001): paper walkthrough and intuition	4
5	Tsitsiklis–Van Roy (2001): paper walkthrough and intuition	4
6	Conceptual comparison of the two papers	5
7	Integrated implementation workflow and code design	5
8	Experimental design	6
9	Results	7
9.1	Shared vanilla benchmark	7
9.2	Shared exercise-policy interpretation	11
9.3	Shared robustness analysis	13
9.4	Shared Asian-style extension	17
9.5	Shared jump-style extension	20
9.6	Direct Longstaff vs TVR comparison	21
10	Implementation and testing	21

11 Critical assessment: strengths, weaknesses, and practical risks	22
12 Scope and claims	23
13 Conclusions	23
14 Reproducibility / code execution	23
15 References	24

1 Introduction

American option pricing is an optimal stopping problem. Unlike a European claim, an American contract can be exercised before maturity, so valuation requires both a price and an exercise rule. That feature makes the problem mathematically richer and computationally more demanding. Even in the case of a vanilla American put under geometric Brownian motion, the value cannot be reduced to a single terminal expectation without solving a continuation-versus-exercise problem over time.

This report studies that problem through two influential papers. Longstaff and Schwartz (2001) provide the main practical algorithm used in the project: least-squares Monte Carlo (LSMC), based on backward induction and continuation-value regression on simulated paths. Tsitsiklis and Van Roy (2001) provide a related but distinct perspective based on approximate dynamic programming and recursive value approximation. The project treats Longstaff–Schwartz as the main empirical pricing method and implements Tsitsiklis–Van Roy as a matched comparator under the same computational testbeds.

The empirical study is organized around a shared synthetic benchmark rather than historical equity calibration. The main benchmark is a vanilla American put under risk-neutral geometric Brownian motion (GBM), priced against an implicit finite-difference American benchmark, a Black–Scholes European lower reference, and a CRR tree check. This shared benchmark is then extended in two directions: an Asian-style extension with multiple state variables and a jump-style extension showing how both regression methods behave when the state dynamics change.

The purpose of the project is to study the two papers with implementation evidence, testing, and critical assessment. It is not a claim of exact numerical reproduction of every published table. The emphasis is on understanding the pricing logic, implementing it in a coherent way, and comparing the two approaches under a common design.

2 Problem setup: American option pricing as an optimal stopping problem

Consider an American put with strike K and maturity T written on an underlying process $(S_t)_{0 \leq t \leq T}$. Under no-arbitrage pricing, the time-zero value under the risk-neutral measure $\mathbb{Q}$ is

$$V_0 = \sup_{\tau \in \mathcal{T}} \mathbb{E}^{\mathbb{Q}} [e^{-r\tau} (K - S_{\tau})^+],$$

where r is the continuously compounded risk-free rate and $\mathcal{T}$ is the set of admissible stopping times. The holder selects the exercise time τ to maximize expected discounted payoff.

In discrete time, with exercise dates $0 = t_0 < t_1 < \dots < t_N = T$, the Bellman recursion is

$$V_i(S_{t_i}) = \max \{h(S_{t_i}), C_i(S_{t_i})\},$$

where

$$h(S_{t_i}) = (K - S_{t_i})^+$$

is the intrinsic value and

$$C_i(S_{t_i}) = \mathbb{E}^{\mathbb{Q}} [e^{-r\Delta t} V_{i+1}(S_{t_{i+1}}) \mid S_{t_i}]$$

is the continuation value. American pricing therefore requires repeated comparison of immediate exercise and the value of waiting.

In low-dimensional settings, this recursion can be handled directly by trees or finite differences. The contribution of both Longstaff–Schwartz and Tsitsiklis–Van Roy is to replace dense state-space discretization with regression-based approximations built from simulated paths.

3 Risk-neutral model and benchmark pricing framework

The project uses a standard Black–Scholes style GBM environment under the risk-neutral measure:

$$dS_t = rS_t dt + \sigma S_t dW_t^{\mathbb{Q}},$$

with constant volatility σ and no dividends. In discrete time,

$$S_{t+\Delta t} = S_t \exp\left(\left(r - \frac{1}{2}\sigma^2\right)\Delta t + \sigma\sqrt{\Delta t}Z\right), \quad Z \sim N(0, 1).$$

The shared synthetic benchmark uses the following grid:

- $K = 40$,
- $r = 0.06$,
- $S_0 \in \{36, 38, 40, 42, 44\}$,
- $\sigma \in \{0.20, 0.40\}$,
- $T \in \{1, 2\}$,
- 50 exercise dates per year,
- 100,000 simulated paths, generated as 50,000 base paths and 50,000 antithetic paths.

This testbed is used for both Longstaff and TVR whenever possible. That common design is important because it allows differences in results to be interpreted as methodological differences rather than differences in case construction.

Three benchmark layers are used in the vanilla study. The Black–Scholes European put is the lower reference,

$$P_{\text{Euro}} = Ke^{-rT}N(-d_2) - S_0N(-d_1),$$

with the standard definitions of d_1 and d_2 . The primary direct American benchmark is an implicit finite-difference solver. A CRR tree is included as an auxiliary check. The finite-difference benchmark is treated as the main direct reference because it solves the same one-dimensional American stopping problem under the same model assumptions.

4 Longstaff–Schwartz (2001): paper walkthrough and intuition

Longstaff and Schwartz (2001) address the basic obstacle in simulation-based American pricing: Monte Carlo methods are well suited to terminal expected values, but American claims require continuation values and exercise decisions at intermediate dates. The paper’s central idea is to approximate continuation values by regression on simulated paths.

At an exercise date t_i , the continuation value is approximated by

$$C_i(S_{t_i}) \approx \hat{C}_i(S_{t_i}) = \sum_{j=0}^m \beta_{i,j} \psi_j(S_{t_i}),$$

where $\{\psi_j\}$ is a chosen basis. In the present implementation, the vanilla benchmark uses the constant and the first three weighted Laguerre polynomials in normalized spot.

The algorithm proceeds by simulating risk-neutral paths, initializing terminal payoffs at maturity, and working backward through the exercise dates. At each date, the regression is carried out on paths that are currently in the money, since those are the states where the exercise decision is relevant. Discounted realized future cash flows are regressed on basis functions of the current state, and exercise is chosen whenever intrinsic value exceeds the fitted continuation value.

That distinction is central. Longstaff regresses realized future cash flows and uses the fitted continuation function to construct a stopping rule. In the revised implementation, the stopping rule is then evaluated on an independent path set rather than only on the fitting sample. This two-pass policy-evaluation step was added because same-path fitting and same-path valuation produced optimistic results in the original benchmark runs.

5 Tsitsiklis–Van Roy (2001): paper walkthrough and intuition

Tsitsiklis and Van Roy (2001) study American-style option pricing from the viewpoint of approximate dynamic programming. Their emphasis is not primarily on regressing future realized cash flows path by path, but on recursively approximating the value function itself.

In discrete time, the Bellman recursion is

$$V_i(s) = \max \left\{ h(s), \mathbb{E}^{\mathbb{Q}} \left[e^{-r\Delta t} V_{i+1}(S_{t_{i+1}}) \mid S_{t_i} = s \right] \right\}.$$

The continuation term is approximated by projection onto a finite-dimensional basis,

$$\hat{C}_i(s) = \sum_{j=0}^m \alpha_{i,j} \phi_j(s),$$

and the value estimate is updated through

$$\hat{V}_i(s) = \max\{h(s), \hat{C}_i(s)\}.$$

This viewpoint makes the role of recursive approximation explicit. Instead of regressing realized future cash flows directly, the method uses discounted next-step estimated values as regression targets. The representative states used in the regressions are the simulated states generated under the pricing model, so the quality of the approximation depends on both the basis specification and the distribution of those simulated states.

In the present project, TVR is implemented as a fitted value iteration under the same testbeds used for Longstaff. After the recursive value approximation is fitted, the implied stopping policy is evaluated on a fresh out-of-sample path set. This policy-evaluation step is necessary if the TVR output is to be compared fairly with Longstaff under a shared benchmark design.

6 Conceptual comparison of the two papers

The two papers solve related problems, but they do so in different ways. Longstaff–Schwartz estimates continuation values by regressing discounted realized future cash flows on the current state and then infers a stopping rule from that comparison. Tsitsiklis–Van Roy uses discounted next-step estimated values and treats the problem more explicitly as approximate value iteration.

That distinction matters for interpretation. In this project, Longstaff–Schwartz serves as the main empirical anchor because it provides the primary practical pricing procedure. TVR serves as a matched comparator under the same computational testbeds. The comparison is therefore methodological: both methods face the same simulated states, payoff structures, exercise grids, and benchmark references, but differ in recursion target and approximation logic.

7 Integrated implementation workflow and code design

The implementation is organized as one shared pricing study with two regression engines. Basis construction, simulation, and direct benchmarking are shared as far as possible. What changes between the two frameworks is the backward recursion itself.

At the basis level, the project uses matched approximation complexity across methods. The vanilla benchmark uses a one-state basis built from weighted Laguerre terms in normalized spot. The Asian-style extension uses a two-state basis in current spot and running arithmetic average, together with selected interaction terms. This matching matters because the comparison should not be driven by one method having a systematically richer approximation space than the other.

At the simulation level, both methods use the same model-side path generators whenever possible. The vanilla study uses GBM with antithetic variates. The Asian-style extension uses the same spot process augmented by a running arithmetic average. The jump-style extension uses a jump-to-ruin process. This shared simulator structure is important because it isolates the effect of the recursion method from the effect of the state process.

The Longstaff engine implements backward induction by regressing discounted realized continuation cash flows on in-the-money paths. The TVR engine implements fitted value iteration by regressing discounted next-step estimated values on current-state basis functions. In both cases, the fitted rule is turned into an implied stopping policy and then evaluated on a fresh path set. That shared policy-evaluation step makes the comparison much more informative than a comparison of fitted time-zero values alone.

The benchmark layer provides the common pricing references in the vanilla study: Black–Scholes European values, finite-difference American values, and CRR tree values. The vanilla scripts for Longstaff and TVR therefore operate under the same benchmark philosophy, and the

Asian-style and jump-style scripts operate under the same extension philosophy. The result is a single workflow with shared testbeds and different recursion logic rather than two disconnected coding exercises.

8 Experimental design

The project is organized around three shared study layers.

The first layer is the vanilla American put benchmark under risk-neutral GBM. This is the main validation exercise. Both Longstaff and TVR are run on the same synthetic grid, under the same exercise-date conventions and path counts, and compared against the same finite-difference benchmark. Black–Scholes European values and CRR tree prices provide supporting references.

The second layer is a shared Asian-style extension. The state includes current spot and running arithmetic average, exercise is prohibited until a lockout date, and the payoff depends on the average rather than spot alone. The purpose is to test how both methods behave when the state vector becomes richer.

The third layer is a shared jump-style extension. The simulator is replaced by a jump-to-ruin process while the pricing logic remains unchanged within each method. This layer is included to show that both regression frameworks can be applied beyond standard GBM.

Robustness analysis is conducted on the vanilla benchmark. On the Longstaff side, the study includes in-sample versus out-of-sample valuation, path sensitivity, exercise-grid sensitivity, basis sensitivity, and seed sensitivity. On the TVR side, the study includes in-sample versus out-of-sample valuation and compact path, timestep, and basis sensitivity exercises. The role of these diagnostics is interpretive as much as numerical: they show how the methods respond to design choices in basis specification, exercise grid, and validation structure.

9 Results

9.1 Shared vanilla benchmark

Table 1: Longstaff–Schwartz vanilla benchmark against the finite-difference American mark.

	spot	sigma	maturity	american_put_fd	american_put_lsmc	lsmc_stderr	lsmc_fit_minus_eval
0	36.0	0.2	1.0	4.4635	4.2058	0.0127	0.2466
1	38.0	0.2	1.0	3.2350	3.2668	0.0112	0.1840
2	40.0	0.2	1.0	2.2995	2.4436	0.0099	0.1341

Table 1: Longstaff–Schwartz vanilla benchmark against the finite-difference American mark.

	spot	sigma	maturity	american_put_fd	american_put_lsmc	lsmc_stderr	lsmc_fit_minus_eval
3	42.0	0.2	1.0	1.6046	1.7456	0.0085	0.0978
4	44.0	0.2	1.0	1.0998	1.1946	0.0071	0.0670
5	36.0	0.2	2.0	4.8280	4.1570	0.0146	0.4995
6	38.0	0.2	2.0	3.7328	3.4155	0.0132	0.4032
7	40.0	0.2	2.0	2.8734	2.7271	0.0119	0.3187
8	42.0	0.2	2.0	2.2024	2.1386	0.0107	0.2546
9	44.0	0.2	2.0	1.6810	1.6561	0.0096	0.1995
10	36.0	0.4	1.0	7.0760	7.2198	0.0212	0.4156
11	38.0	0.4	1.0	6.1216	6.3681	0.0202	0.3616
12	40.0	0.4	1.0	5.2861	5.4879	0.0192	0.3057
13	42.0	0.4	1.0	4.5576	4.7839	0.0183	0.2699
14	44.0	0.4	1.0	3.9240	4.1321	0.0172	0.2311
15	36.0	0.4	2.0	8.4868	8.3510	0.0250	1.0208
16	38.0	0.4	2.0	7.6478	7.6528	0.0242	0.9240
17	40.0	0.4	2.0	6.8969	6.9714	0.0235	0.8526
18	42.0	0.4	2.0	6.2246	6.2554	0.0228	0.7495
19	44.0	0.4	2.0	5.6219	5.6665	0.0221	0.6788

Table 1 shows that the revised Longstaff benchmark produces American put values of the correct order of magnitude across the full parameter grid. The reported two-pass values are generally closer to the finite-difference benchmark than the original same-path values, but the gap does not disappear uniformly. In some cases the method remains above the benchmark, while in others it falls below it. The column recording the fit-versus-evaluation gap is particularly informative because it quantifies the effect of same-path optimism in the original implementation.

Table 2: Tsitsiklis–Van Roy vanilla benchmark under the same syntax.

	spot	sigma	maturity	american_put_fd	tvr_price_in_sample	tvr_price_out_of_sample	tvr_stderr
0	36.0	0.2	1.0	4.4635	4.3861	4.3851	0.0114
1	38.0	0.2	1.0	3.2350	3.1977	3.1925	0.0106
2	40.0	0.2	1.0	2.2995	2.2864	2.2783	0.0094
3	42.0	0.2	1.0	1.6046	1.5999	1.5957	0.0082
4	44.0	0.2	1.0	1.0998	1.0966	1.0912	0.0070
5	36.0	0.2	2.0	4.8280	4.6962	4.7065	0.0135
6	38.0	0.2	2.0	3.7328	3.6604	3.6598	0.0127
7	40.0	0.2	2.0	2.8734	2.8240	2.8285	0.0118

Table 2: Tsitsiklis–Van Roy vanilla benchmark under the same synt

	spot	sigma	maturity	american_put_fd	tvr_price_in_sample	tvr_price_out_of_sample	tvr_s
8	42.0	0.2	2.0	2.2024	2.1582	2.1582	0.0107
9	44.0	0.2	2.0	1.6810	1.6368	1.6378	0.0097
10	36.0	0.4	1.0	7.0760	6.9015	6.9009	0.0216
11	38.0	0.4	1.0	6.1216	5.9237	5.9223	0.0210
12	40.0	0.4	1.0	5.2861	5.1318	5.1268	0.0200
13	42.0	0.4	1.0	4.5576	4.4370	4.4366	0.0189
14	44.0	0.4	1.0	3.9240	3.8354	3.8380	0.0178
15	36.0	0.4	2.0	8.4868	8.4146	8.4300	0.0247
16	38.0	0.4	2.0	7.6478	7.5979	7.6128	0.0242
17	40.0	0.4	2.0	6.8969	6.8597	6.8591	0.0236
18	42.0	0.4	2.0	6.2246	6.2001	6.1972	0.0230
19	44.0	0.4	2.0	5.6219	5.5991	5.5954	0.0223

Table 2 places TVR under the same benchmark conditions. The out-of-sample TVR values are generally close to the finite-difference price and are often slightly below it. This is a useful result because it shows that the revised TVR implementation is no longer behaving like the older loose prototype that materially overstated vanilla prices. The final comparison is therefore between two benchmarked regression procedures rather than between one benchmarked method and one poorly aligned comparator.

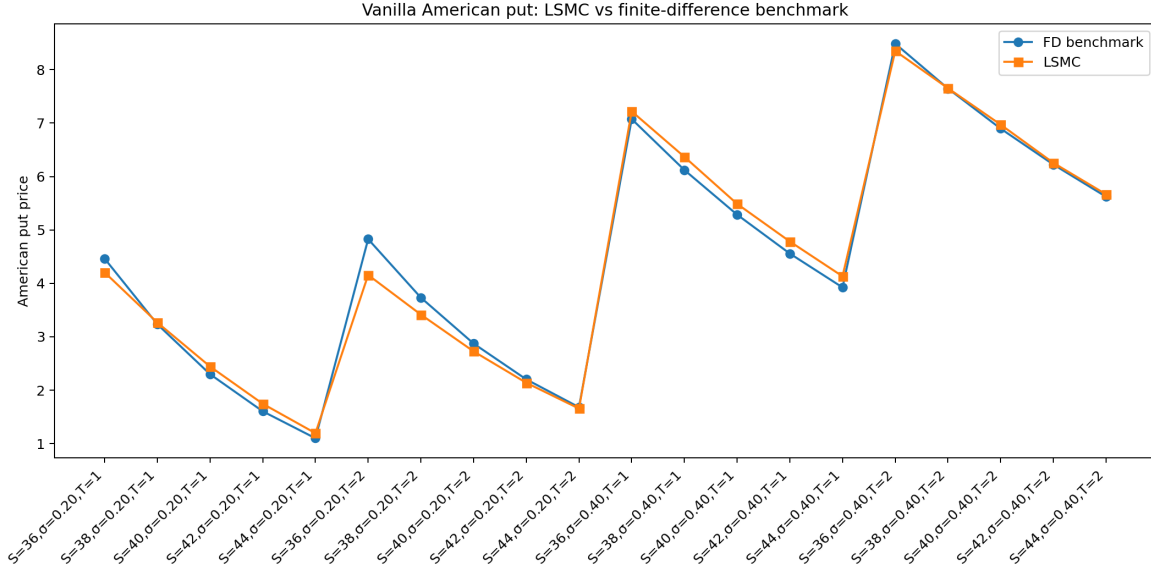


Figure 1: Vanilla benchmark comparison: Longstaff LSMC versus finite-difference benchmark.

Figure 1 shows the Longstaff side of the shared vanilla benchmark. The visual pattern matches the table: the revised benchmark is much tighter than the earlier same-path version, but there is still visible case-by-case variation around the finite-difference reference. That variation is part of the substantive numerical story of the method rather than a formatting artifact.

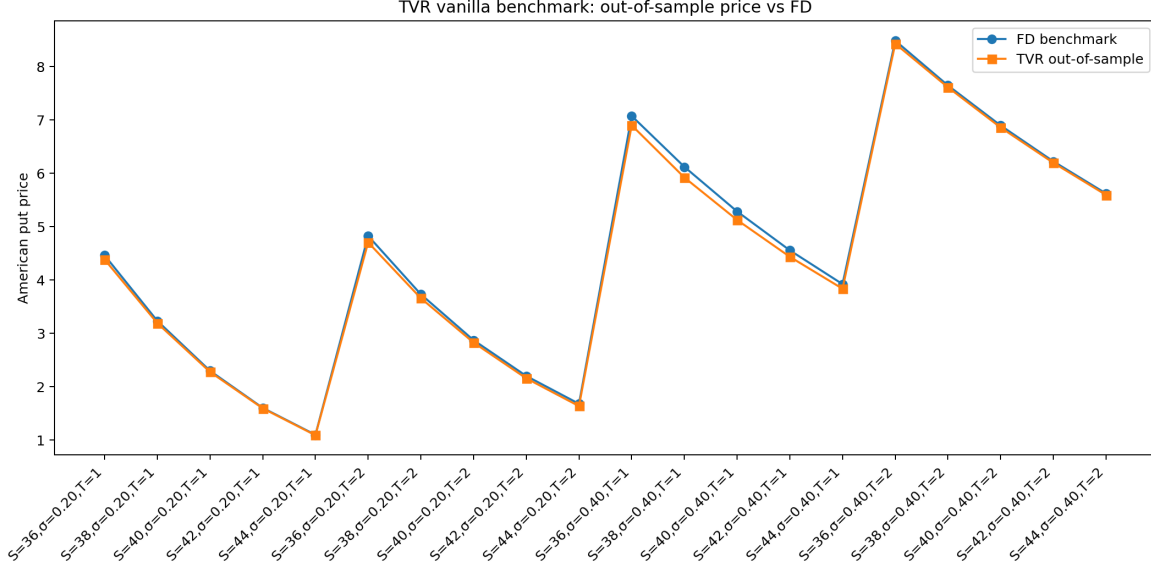


Figure 2: Vanilla benchmark comparison: TVR out-of-sample value versus finite-difference benchmark.

Figure 2 shows the corresponding TVR benchmark. The TVR values cluster closely around the finite-difference benchmark across the shared grid. Read together with Table 2, the figure suggests that the revised TVR comparator is reasonably well aligned to the common testbed and can be interpreted as a genuine methodological alternative rather than a separate ad hoc calculation.

9.2 Shared exercise-policy interpretation

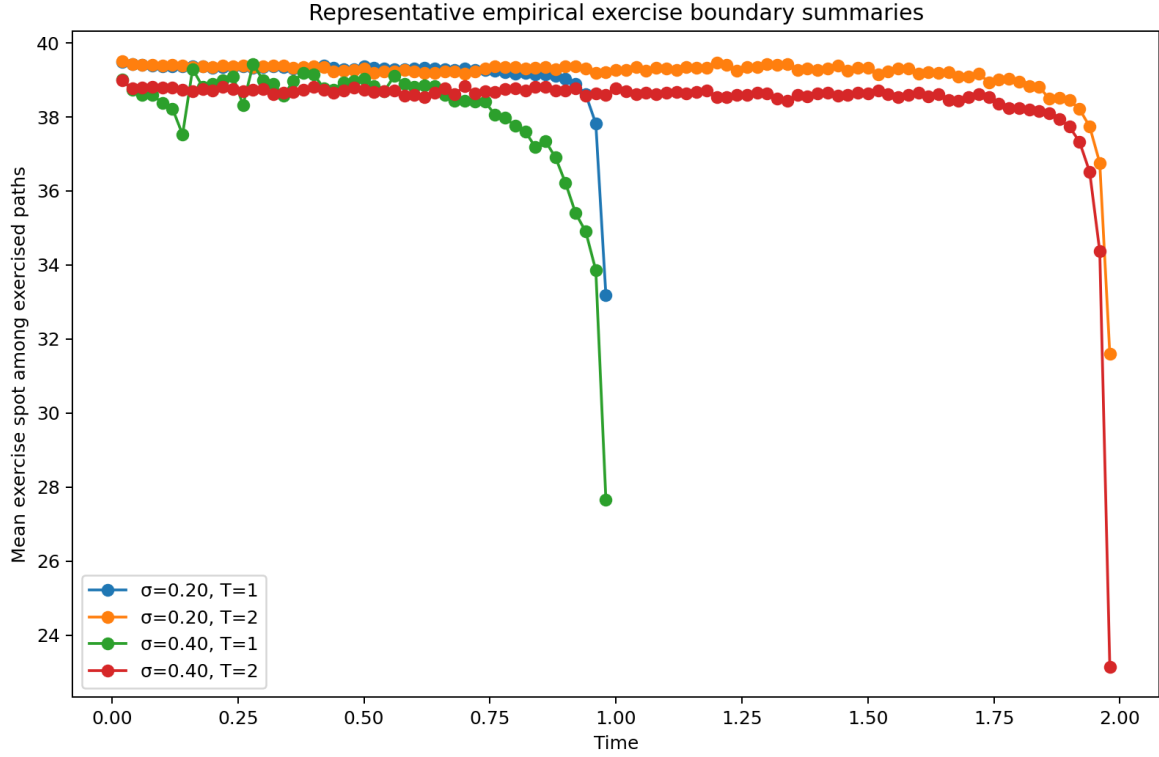


Figure 3: Empirical exercise-boundary summary: Longstaff vanilla benchmark.

Figure 3 summarizes exercise behavior for representative Longstaff vanilla cases. It should not be read as an exact free boundary. Rather, it is an empirical summary of states at which exercise occurs on simulated paths. The figure is still useful because it shows where the estimated stopping rule becomes active and how that region shifts across maturity and volatility settings.

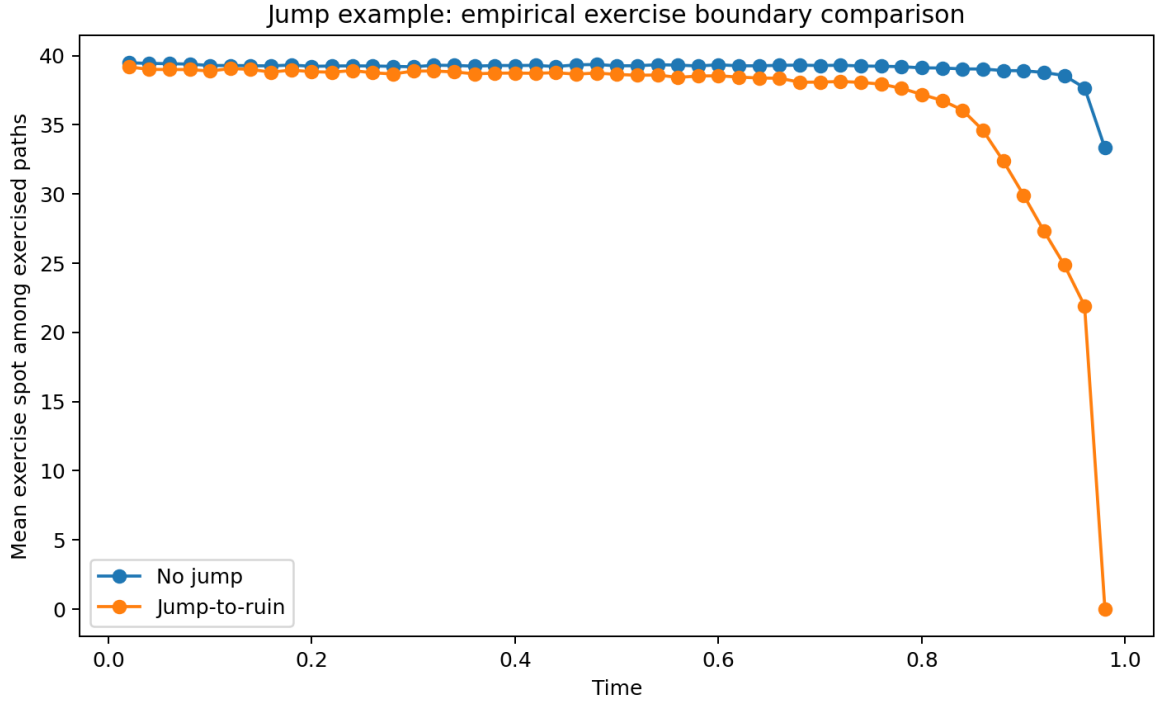


Figure 4: Jump-style boundary comparison: Longstaff.

The Longstaff jump-style boundary figure extends the same idea to the jump-to-ruin setting. Once downward jump risk is introduced, exercise decisions occur under very different state dynamics. The figure should again be read as a summary of simulated exercise behavior rather than as a structural boundary, but it makes the change in stopping behavior visible.

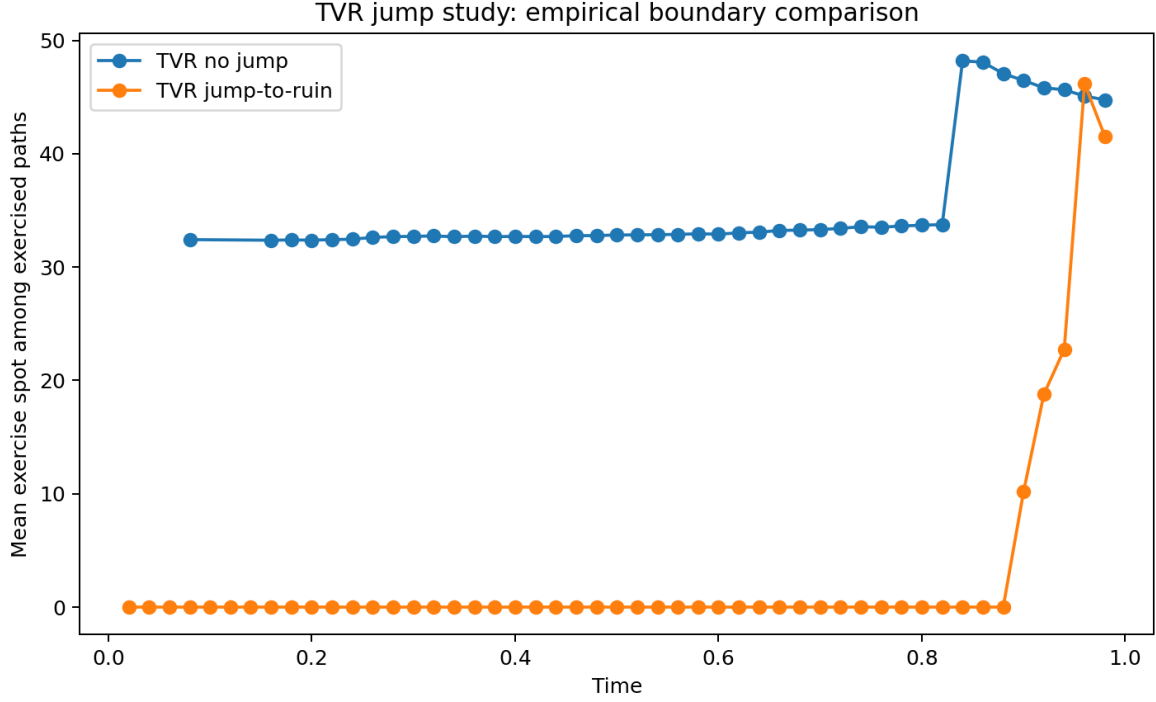


Figure 5: Jump-style boundary comparison: TVR.

Figure 5 provides the analogous TVR boundary summary. The qualitative pattern is similar: exercise tends to occur in lower stock-price regions, and the jump case alters the timing and location of exercise. The key methodological point is that both engines generate interpretable stopping behavior under the same changed dynamics.

9.3 Shared robustness analysis

Table 3: Longstaff in-sample versus out-of-sample p cases.

	spot	sigma	maturity	fit_seed	eval_seed	in_sample_price	in_sample_stderr	out_of_sample
0	36.0	0.2	1.0	20260406	20260406	4.4524	0.0135	4.2057
1	36.0	0.2	1.0	20260407	20260407	4.4517	0.0135	4.2050
2	36.0	0.2	1.0	20260408	20260408	4.4519	0.0135	4.2052
3	40.0	0.2	2.0	20261406	20262406	3.0596	0.0134	2.7442
4	40.0	0.2	2.0	20261407	20262407	3.0275	0.0133	2.7219
5	40.0	0.2	2.0	20261408	20262408	3.0201	0.0134	2.6921

Table 3: Longstaff in-sample versus out-of-sample p cases.

	spot	sigma	maturity	fit_seed	eval_seed	in_sample_price	in_sample_stderr	out_of_sample
6	44.0	0.4	1.0	20262406	20264406	4.3604	0.0183	4.1103
7	44.0	0.4	1.0	20262407	20264407	4.3908	0.0184	4.1526
8	44.0	0.4	1.0	20262408	20264408	4.3708	0.0183	4.1197
9	40.0	0.4	2.0	20263406	20266406	7.8097	0.0265	6.9652
10	40.0	0.4	2.0	20263407	20266407	7.6596	0.0265	6.8219
11	40.0	0.4	2.0	20263408	20266408	7.8020	0.0265	6.9743

Table 4: TVR in-sample versus out-of-sample policy valuation for representative vanilla cas

	spot	sigma	maturity	fit_seed	eval_seed	in_sample_price	out_of_sample_price	difference
0	36.0	0.2	1.0	20260407	20265407	4.3861	4.3810	0.0050
1	36.0	0.2	1.0	20260408	20265408	4.4016	4.3894	0.0122
2	36.0	0.2	1.0	20260409	20265409	4.3960	4.3882	0.0078
3	40.0	0.2	2.0	20261407	20267407	2.8035	2.8173	-0.0138
4	40.0	0.2	2.0	20261408	20267408	2.8148	2.8109	0.0039
5	40.0	0.2	2.0	20261409	20267409	2.8219	2.8296	-0.0077
6	44.0	0.4	1.0	20262407	20269407	3.8648	3.8395	0.0253
7	44.0	0.4	1.0	20262408	20269408	3.8555	3.8601	-0.0046
8	44.0	0.4	1.0	20262409	20269409	3.8523	3.8571	-0.0048
9	40.0	0.4	2.0	20263407	20271407	6.8432	6.8289	0.0143
10	40.0	0.4	2.0	20263408	20271408	6.8294	6.8507	-0.0214
11	40.0	0.4	2.0	20263409	20271409	6.8481	6.8600	-0.0119

Table 3 and Table 4 report the central fit-versus-evaluation diagnostic for the two methods. On the Longstaff side, the gap between in-sample and out-of-sample values is clearly visible and becomes substantial in the harder high-volatility, long-maturity case. On the TVR side, the same diagnostic is much tighter in the representative cases shown here. The contrast does not imply that TVR is categorically better; it shows that the two recursion schemes react differently to the same validation design.

Table 5: Longstaff path-count sensitivity under the shared vanilla benchmark.

	spot	sigma	maturity	paths_total	lsmc_price	lsmc_stderr	fd_benchmark	lsmc_minus_fd
0	40.0	0.2	1.0	25000	2.3174	0.0194	2.2995	0.0179
1	40.0	0.2	1.0	50000	2.3878	0.0139	2.2995	0.0884
2	40.0	0.2	1.0	100000	2.4472	0.0099	2.2995	0.1477

Table 5: Longstaff path-count sensitivity under the shared vanilla benchmark.

	spot	sigma	maturity	paths_total	lsmc_price	lsmc_stderr	fd_benchmark	lsmc_minus_fd
3	40.0	0.2	1.0	200000	2.4474	0.0070	2.2995	0.1479
4	40.0	0.4	2.0	25000	6.9352	0.0467	6.8969	0.0383
5	40.0	0.4	2.0	50000	6.8354	0.0333	6.8969	-0.0615
6	40.0	0.4	2.0	100000	6.8574	0.0235	6.8969	-0.0395
7	40.0	0.4	2.0	200000	6.9576	0.0166	6.8969	0.0607

Table 6: TVR path-count sensitivity under the shared vanilla benchmark.

	spot	sigma	maturity	paths_total	tvr_price_out_of_sample	tvr_stderr_out_of_sample	fd_ben
0	40.0	0.2	1.0	25000	2.2703	0.0191	2.2995
1	40.0	0.2	1.0	50000	2.2990	0.0134	2.2995
2	40.0	0.2	1.0	100000	2.2909	0.0094	2.2995
3	40.0	0.2	1.0	200000	2.2818	0.0067	2.2995
4	40.0	0.4	2.0	25000	6.8895	0.0472	6.8969
5	40.0	0.4	2.0	50000	6.8606	0.0334	6.8969
6	40.0	0.4	2.0	100000	6.8318	0.0236	6.8969
7	40.0	0.4	2.0	200000	6.8500	0.0167	6.8969

Table 5 and Table 6 show that increasing path count reduces standard error, but does not by itself guarantee benchmark convergence. The Longstaff values move around the finite-difference benchmark in a case-dependent way, while TVR stays relatively close to it across the tested path counts. This comparison reinforces the idea that path count is only one element of performance; recursion target and policy construction matter as well.

Table 7: Longstaff exercise-grid sensitivity under the shared vanilla benchmark.

	spot	sigma	maturity	exercise_dates_per_year	lsmc_price	lsmc_stderr	fd_benchmark	lsmc_m
0	40.0	0.2	1.0	25	2.5610	0.0097	2.2821	0.2789
1	40.0	0.2	1.0	50	2.4399	0.0098	2.2995	0.1404
2	40.0	0.2	1.0	100	2.3439	0.0100	2.3086	0.0353
3	40.0	0.4	2.0	25	7.2098	0.0230	6.8724	0.3373
4	40.0	0.4	2.0	50	6.8366	0.0235	6.8969	-0.0603
5	40.0	0.4	2.0	100	6.8272	0.0238	6.9096	-0.0824

Table 8: TVR exercise-grid sensitivity under the shared vanilla benchmark.

	spot	sigma	maturity	exercise_dates_per_year	tvr_price_out_of_sample	tvr_stderr_out_of_sample
0	40.0	0.2	1.0	25	2.2954	0.0093
1	40.0	0.2	1.0	50	2.2906	0.0094
2	40.0	0.2	1.0	100	2.2809	0.0095
3	40.0	0.4	2.0	25	6.8635	0.0233
4	40.0	0.4	2.0	50	6.8611	0.0236
5	40.0	0.4	2.0	100	6.8189	0.0239

Table 7 and Table 8 show that both methods are sensitive to the exercise grid. Longstaff can shift from benchmark overvaluation to benchmark undervaluation as the number of exercise dates changes. TVR remains comparatively close to the benchmark in the selected cases, but the effect of the grid is still visible. The main lesson is that discretization is part of the modeling design, not a neutral background detail.

Table 9: Longstaff basis-size sensitivity under the shared vanilla benchmark.

	spot	sigma	maturity	basis_size	lsmc_price	lsmc_stderr	fd_benchmark	lsmc_minus_fd
0	40.0	0.2	1.0	2	2.3846	0.0099	2.2995	0.0851
1	40.0	0.2	1.0	3	2.4432	0.0099	2.2995	0.1438
2	40.0	0.2	1.0	4	2.4327	0.0099	2.2995	0.1332
3	40.0	0.2	1.0	5	2.4576	0.0098	2.2995	0.1582
4	40.0	0.4	2.0	2	6.8897	0.0235	6.8969	-0.0072
5	40.0	0.4	2.0	3	7.0039	0.0236	6.8969	0.1070
6	40.0	0.4	2.0	4	6.9752	0.0234	6.8969	0.0783
7	40.0	0.4	2.0	5	6.9922	0.0234	6.8969	0.0953

Table 10: TVR basis-size sensitivity under the shared vanilla benchmark.

	spot	sigma	maturity	basis_size	tvr_price_out_of_sample	tvr_stderr_out_of_sample	fd_benchmark
0	40.0	0.2	1.0	2	2.1510	0.0105	2.2995
1	40.0	0.2	1.0	3	2.2576	0.0099	2.2995
2	40.0	0.2	1.0	4	2.2885	0.0094	2.2995
3	40.0	0.2	1.0	5	2.2850	0.0096	2.2995
4	40.0	0.4	2.0	2	6.4664	0.0253	6.8969
5	40.0	0.4	2.0	3	6.7995	0.0244	6.8969
6	40.0	0.4	2.0	4	6.8352	0.0236	6.8969
7	40.0	0.4	2.0	5	6.8621	0.0233	6.8969

Table 9 and Table 10 show that accuracy is not monotone in the number of basis functions. A richer basis can improve approximation in principle, but it can also amplify instability in finite samples. This pattern appears for both methods, though the strength of the effect differs. The comparison supports the broader point that basis design is a substantive modeling choice rather than a purely technical parameter.

Table 11: Longstaff seed-level results for representative vanilla benchmark cases.

	spot	sigma	maturity	seed	lsmc_price	lsmc_stderr	fd_benchmark	bias_vs_fd
0	36.0	0.2	1.0	20260406	4.2058	0.0127	4.4635	-0.2577
1	36.0	0.2	1.0	20260407	4.1992	0.0127	4.4635	-0.2643
2	36.0	0.2	1.0	20260408	4.2122	0.0127	4.4635	-0.2513
3	36.0	0.2	1.0	20260409	4.2014	0.0127	4.4635	-0.2621
4	36.0	0.2	1.0	20260410	4.2201	0.0127	4.4635	-0.2433
6	40.0	0.2	2.0	20260406	2.7271	0.0119	2.8734	-0.1463
7	40.0	0.2	2.0	20260407	2.7481	0.0119	2.8734	-0.1253
8	40.0	0.2	2.0	20260408	2.7541	0.0119	2.8734	-0.1193
9	40.0	0.2	2.0	20260409	2.7243	0.0119	2.8734	-0.1491
10	40.0	0.2	2.0	20260410	2.7545	0.0120	2.8734	-0.1188
12	44.0	0.4	1.0	20260406	4.1321	0.0172	3.9240	0.2081
13	44.0	0.4	1.0	20260407	4.1841	0.0173	3.9240	0.2601
14	44.0	0.4	1.0	20260408	4.1744	0.0173	3.9240	0.2504
15	44.0	0.4	1.0	20260409	4.1210	0.0171	3.9240	0.1970
16	44.0	0.4	1.0	20260410	4.1425	0.0172	3.9240	0.2185

Table 12: Longstaff summary across repeated seeds for representative vanilla benchmark cases.

	case	mean_lsm_price	sd_across_runs	fd_benchmark	average_bias_vs_fd
5	S=36.0, $\sigma=0.2$, T=1.0	4.2077	0.0085	4.4635	-0.2557
11	S=40.0, $\sigma=0.2$, T=2.0	2.7416	0.0148	2.8734	-0.1318
17	S=44.0, $\sigma=0.4$, T=1.0	4.1508	0.0273	3.9240	0.2268

Table 11 and Table 12 separate the repeated-seed output into seed-level runs and case-level summaries. This avoids mixing literal summary rows into the same seed column as if they were additional replications. The seed-level table shows the dispersion across runs, while the summary table reports the mean LSM price, the standard deviation across runs, and the average bias relative to the finite-difference benchmark. Together, they make clear that Monte Carlo variation remains present even after the larger workflow revisions.

9.4 Shared Asian-style extension

	example	spot0	strike	risk_free	sigma	maturity	lockout_time	exercise_dates
0	American-Bermuda-Asian	40.0	40.0	0.06	0.2	2.0	0.25	100

Table 13 shows the Longstaff Asian-style extension under the shared design. The extension introduces a second state variable—the running arithmetic average—and a lockout period before exercise is allowed. The resulting price is therefore not directly comparable to the vanilla benchmark, but it demonstrates that the continuation regression can accommodate a richer state description within the same backward-induction framework.

	example	spot0	strike	risk_free	sigma	maturity	lockout_time	exercise_dates
0	TVR Asian shared extension	40.0	40.0	0.06	0.2	2.0	0.25	100

Table 14 presents the corresponding TVR result. The out-of-sample TVR price exceeds its European-style Monte Carlo reference by a visible margin, indicating that the approximate dynamic-programming recursion is also capturing an early-exercise premium in the multi-state case. As on the Longstaff side, this is a compact shared extension study rather than a direct PDE-benchmarked replication.

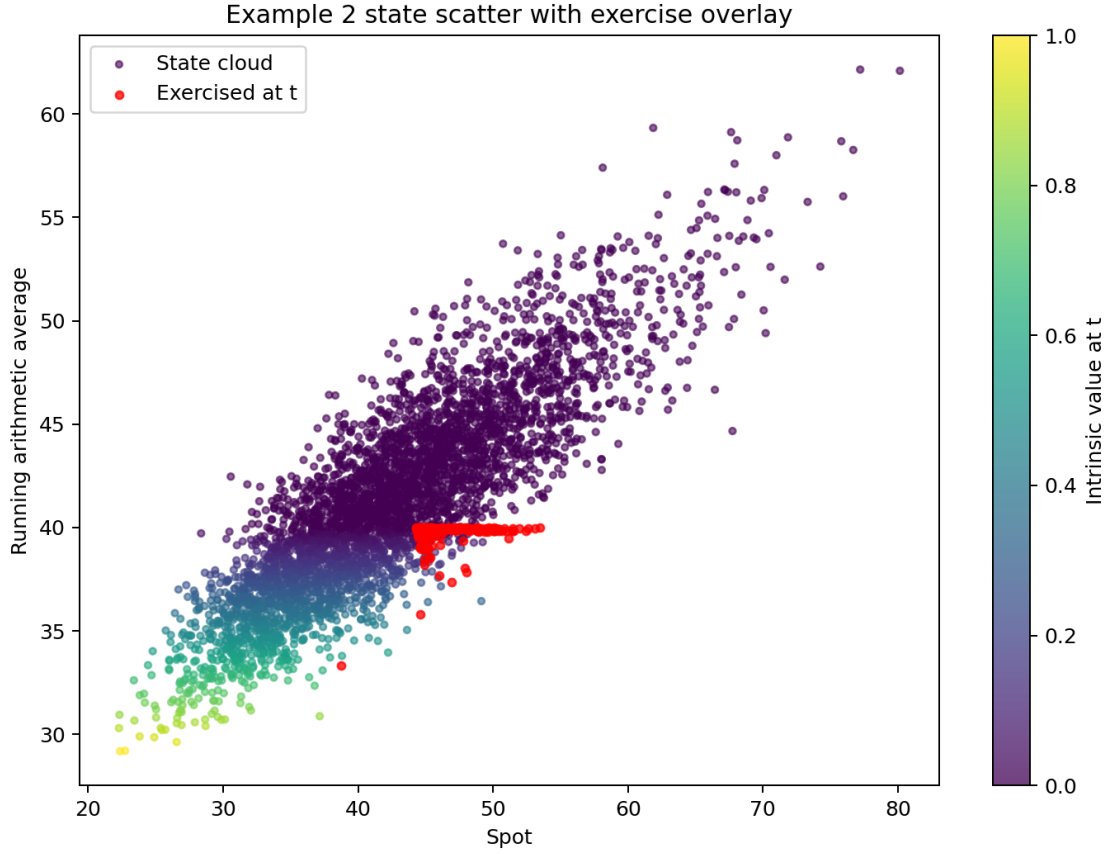


Figure 6: Asian-style state cloud with exercise overlay: Longstaff.

Figure 6 shows the Longstaff state cloud in spot and running average, together with the exercise overlay. The figure is useful because it makes the two-state nature of the exercise problem visible. Exercise is no longer a function of spot alone; it depends on the interaction between current level and path history through the running average.

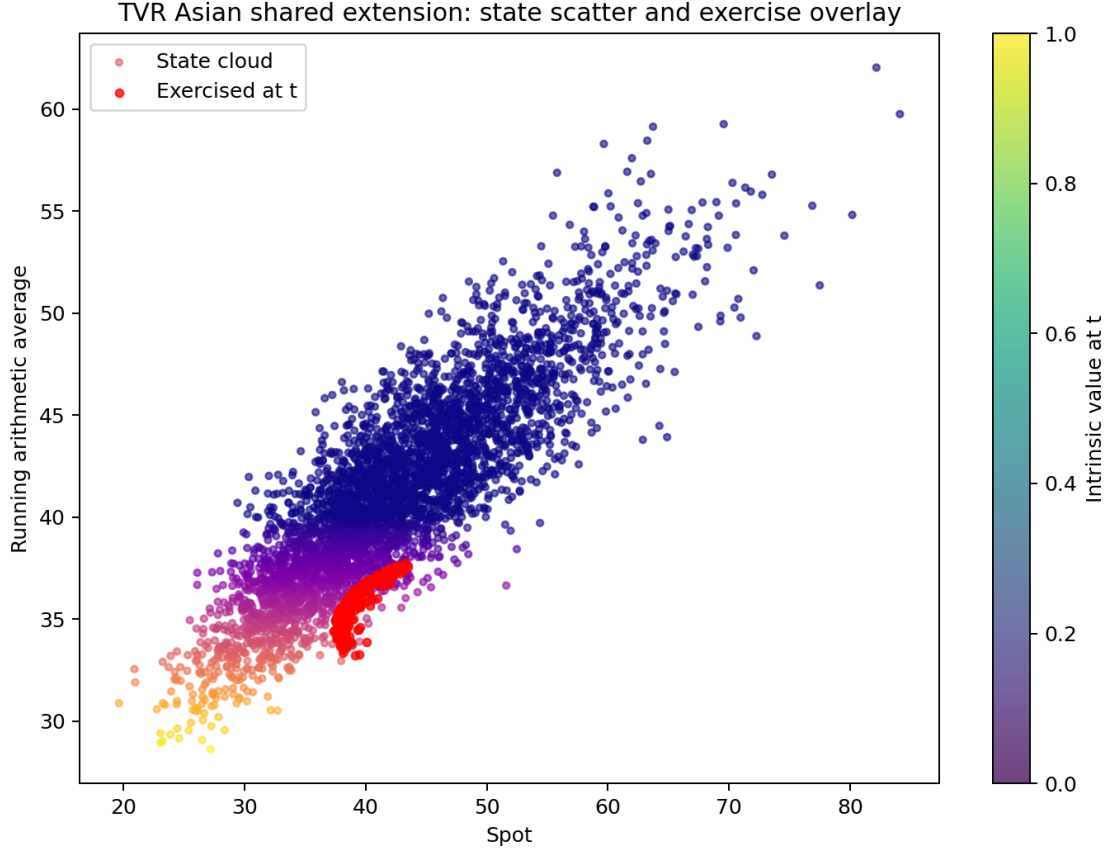


Figure 7: Asian-style state cloud with exercise overlay: TVR.

Figure 7 provides the analogous TVR visualization. The structure is similar, although the implied exercise pattern is not identical. Read together, the two figures show that both methods can operate on the same richer state space while retaining different recursive logic.

9.5 Shared jump-style extension

Table 15: Jump-style extension: Longstaff results.

	model	spot0	strike	sigma	risk_free	maturity	jump_intensity	american_lsm	lsm_sto
0	no_jump_gbm	40.0	40.0	0.2	0.06	1.0	0.0	2.5657	0.0104
1	jump_to_ruin	40.0	40.0	0.2	0.06	1.0	0.4	14.9459	0.0556

Table 15 shows the Longstaff jump-style extension. Once jump-to-ruin risk is introduced, both

American and European put values rise sharply relative to the no-jump case because the stock can collapse to zero before maturity. The early-exercise premium remains positive, but its scale differs from the vanilla benchmark because the underlying risk environment has changed.

Table 16: Jump-style extension

	model	spot0	strike	sigma	risk_free	maturity	jump_intensity	tvr_price_in_sample
0	no_jump_gbm	40.0	40.0	0.2	0.06	1.0	0.0	2.2778
1	jump_to_ruin	40.0	40.0	0.2	0.06	1.0	0.4	14.1417

Table 16 shows the same qualitative effect under TVR. The out-of-sample jump-to-ruin value is far above the no-jump value, and the early-exercise premium remains positive. The two methods therefore agree on the broad economic effect of introducing catastrophic downward jump risk, even though their recursive approximations differ.

The boundary figures in Figure 4 and Figure 5 support the same reading visually. In each case, the pricing workflow remains intact while only the simulator changes. This is the main point of the jump-style study: it demonstrates modularity with respect to the state dynamics.

9.6 Direct Longstaff vs TVR comparison

After the shared evidence has been laid out, the comparison between Longstaff and TVR can be summarized in four points. First, the recursion target differs: Longstaff regresses discounted realized future cash flows, whereas TVR regresses discounted next-step estimated values. Second, the benchmark behavior differs: in the revised vanilla study, Longstaff may sit above or below the finite-difference benchmark depending on the case, while TVR tends to remain close to it and is often slightly below it. Third, the validation behavior differs: Longstaff shows visible same-path optimism unless the policy is evaluated out of sample, while TVR's matched implementation shows a smaller in-sample versus out-of-sample gap in the representative cases examined here. Fourth, the interpretation differs: Longstaff is more directly tied to continuation regression on in-the-money paths, while TVR more clearly reflects approximate dynamic programming through recursive value approximation.

10 Implementation and testing

The implementation and testing logic is driven by the course rubric rather than by code structure alone. Two pricing frameworks were implemented under a common benchmark philosophy: Longstaff as the main practical algorithm and TVR as a matched approximate-dynamic-programming comparator. Both were tested on the same synthetic vanilla benchmark, the same Asian-style extension, and the same jump-style extension.

The vanilla case was directly benchmarked. For that setting, Black–Scholes provides a lower reference, finite differences provide the main American benchmark, and CRR provides an auxiliary check. The project therefore does more than compare one regression method to another: it compares both methods to a direct pricing reference under the same one-dimensional model.

The extension cases were handled differently. The Asian-style and jump-style studies are included as method extensions rather than as fully benchmarked replications. They show how the recursion methods behave when the state vector or the simulator changes, but they are not meant to carry the same direct-benchmarking burden as the vanilla put.

Testing also relied on in-sample versus out-of-sample diagnostics. On the Longstaff side, that diagnostic revealed clear same-path optimism and motivated the revised two-pass policy evaluation. On the TVR side, the same diagnostic was implemented so that the comparator would be judged under the same validation principle. This shared-comparison design makes the results easier to interpret because both methods are examined under comparable rules.

11 Critical assessment: strengths, weaknesses, and practical risks

The project highlights the strengths of both papers. Longstaff–Schwartz offers a direct and practical method for constructing an exercise rule from simulated cash flows. It is especially useful when the state process is rich enough that direct grids become inconvenient. Tsitsiklis–Van Roy provides a useful approximate-dynamic-programming perspective and emphasizes the recursive nature of value approximation. Together, the two methods illustrate two closely related ways of handling continuation values in American pricing.

The project also makes clear that both methods are numerically sensitive. Basis choice matters. Exercise-grid design matters. Validation design matters. The Longstaff results show that same-path pricing can be materially optimistic. The TVR results show that fitted value iteration can perform reasonably under the shared testbed, but its stability also depends on the chosen approximation space and on the distribution of simulated representative states.

These issues matter in practice. If regression-based American pricing methods are used carelessly, institutions can misprice early-exercise features through poor basis specification, unstable regression fits, weak benchmarking, or reliance on in-sample performance. The project does not claim to solve those governance problems. What it does show is that they arise naturally even in a controlled academic study.

Several simplifications remain. The finite-difference solver is a one-dimensional benchmark implementation rather than a production PDE framework. The Asian-style extension has no direct PDE benchmark. The jump process is stylized. Neither pricing framework is presented as a production library. These limitations should be kept in mind when interpreting the results.

12 Scope and claims

The scope of the project is limited and should be stated plainly. This is a study of the Longstaff–Schwartz and Tsitsiklis–Van Roy papers with computational evidence under shared testbeds. Longstaff is implemented as the main practical pricing algorithm. TVR is implemented as a matched comparator under the same simulated cases and benchmark philosophy.

The project does not claim exact reproduction of every numerical example in either paper. The Asian-style and jump-style studies are shared extensions rather than exact replications of published tables. The project does not calibrate to market implied-volatility surfaces, does not incorporate a full production market-data workflow, and does not attempt to build a production pricing library.

Within that scope, the claims are clear. The project shows how the two papers can be understood through a common computational framework, how their recursion logic differs, and how those differences appear in benchmark alignment, out-of-sample behavior, and robustness.

13 Conclusions

This report studies American option pricing as an optimal stopping problem through two related regression-based frameworks. Longstaff–Schwartz serves as the main empirical pricing method, and Tsitsiklis–Van Roy serves as a matched approximate-dynamic-programming comparator under the same computational design.

The shared vanilla benchmark shows that both methods can produce reasonable American put values under a common GBM testbed, but that their benchmark behavior and validation sensitivity differ. Longstaff’s continuation regression on realized future cash flows leads to visible same-path optimism unless the policy is evaluated on a fresh sample. TVR’s fitted value iteration, when implemented under the same design and evaluated out of sample, produces values that are generally close to the finite-difference benchmark in the tested cases.

The shared Asian-style and jump-style studies show that both methods extend beyond the one-dimensional vanilla setting. The broader lesson is that regression-based American pricing is flexible, but its numerical performance depends on basis design, state representation, exercise-grid choice, and the way the stopping policy is validated.

14 Reproducibility / code execution

The Longstaff workflow is executed from the project root using the scripts under `src/LongstaffMethod`:

```
python src/LongstaffMethod/run_longstaff_vanilla.py
python src/LongstaffMethod/run_longstaff_asian.py
python src/LongstaffMethod/run_longstaff_jump.py
```

The TVR workflow is executed from the project root using the scripts under `src/TVR`:

```
python src/TVR/run_tvr_vanilla.py
python src/TVR/run_tvr_asian.py
python src/TVR/run_tvr_jump.py
```

Outputs are written to `outputs/`, visuals to `visuals/`, and revision notes to `SupportFiles/`. The vanilla bias note for Longstaff is stored in `SupportFiles/longstaff_vanilla_bias_fix_log.txt`.

The report can be rendered using Quarto:

```
quarto render report/american_options_lsmc_study.qmd --to pdf
```

If PDF rendering is unavailable, HTML output can be generated instead:

```
quarto render report/american_options_lsmc_study.qmd --to html
```

15 References

- Longstaff, F. A., & Schwartz, E. S. (2001). Valuing American options by simulation: A simple least-squares approach. *The Review of Financial Studies*, 14(1), 113–147.
- Tsitsiklis, J. N., & Van Roy, B. (2001). Regression methods for pricing complex American-style options. *IEEE Transactions on Neural Networks*, 12(4), 694–703.